



Práctica 2: Tipos de algoritmos

Algorítmica

Víctor M. Campello

Curso 2025/2026

Descripción

En esta práctica vais a implementar y analizar varios tipos de algoritmos que hemos visto en clase: divide y vencerás, voraz o basado en programación dinámica.

Divide y vencerás

Cuando calculamos el mínimo de una lista de n elementos, debemos hacer $n-1$ comparaciones para asegurarnos que hemos encontrado el mínimo. Lo mismo ocurre para el máximo. Un algoritmo para determinar el mínimo es el siguiente:

```
function MÍNIMO( $A$ )  
   $min = A[1]$   
  for  $i = 2, A.length$  do  
    if  $min > A[i]$  then  
       $min = A[i]$   
    end if  
  end for  
  return  $min$   
end function
```

Es decir, el algoritmo es $\Theta(n)$ y es óptimo. No obstante, podemos obtener el mínimo (o máximo) de una lista mediante un algoritmo divide y vencerás que se ejecute en hilos paralelos de un procesador.

Así mismo, es posible calcular simultáneamente el máximo y el mínimo usando como

mucho $3\lfloor n/2 \rfloor$ comparaciones.

Programación dinámica

El siguiente problema a resolver mediante un algoritmo de programación dinámica se denomina el problema de la subsecuencia común más larga. Es decir, dadas dos secuencias del tipo

$$\begin{aligned}S_1 &= ACCGGTTCGAGTGCGCGGAAGCCGGCCGAA \\S_2 &= GTCGTTCGGAATGCCGTTGCTCTGTAAA\end{aligned}$$

se debería encontrar $S_3 = GTCGTTCGGAAGCCGGCCGAA$. Es útil para identificar subcadenas comunes en secuencias genéticas de organismos. Los detalles del método los investigaréis por vuestra cuenta.

Algoritmo voraz

El último problema consiste en optimización de una memoria caché. La memoria caché consiste en un almacenamiento de datos que pueden ser recuperados más rápidamente que en un disco duro habitual. Cuando un programa se ejecuta, hace una serie de peticiones de memoria $\langle r_1, r_2, \dots, r_n \rangle$, donde cada petición corresponde a un dato particular. Por ejemplo, suponed que un programa accede a cuatro posibles elementos $\{a, b, c, d\}$. Una posible secuencia de peticiones podría ser $\langle d, b, d, b, d, a, c, d, b, a, c, b \rangle$. Si k es el tamaño de la memoria caché, cuando el programa accede al elemento $(k + 1)$, el sistema gestor de la caché debe decidir qué k elementos mantener en memoria. Es decir, para cada petición r_i , el algoritmo que gestiona la caché comprueba si r_i está guardado entre los k elementos. Si lo está, tenemos un **acierto en caché**. Si no lo está, tendremos un **fallo en caché**. Si no se encuentra en la caché, el sistema recupera r_i de la memoria general y el algoritmo gestor de la caché debe decidir si mantiene r_i en caché. Si lo mantiene, debe decidir qué elemento eliminar de la caché. El objetivo será minimizar el número de fallos en caché sobre la secuencia de peticiones.

Habitualmente, la consulta en caché es un proceso en directo, pero nosotros vamos a considerar el problema en diferido. Es decir, dada una secuencia de peticiones futuras, qué elementos guardamos y qué cambios hacemos para minimizar los fallos. Una estrategia voraz para resolver el problema consiste en eliminar en cada paso necesario aquel elemento que se pida lo más tarde posible en la secuencia de peticiones. Es decir, eliminar de caché aquel elemento que se acceda más tarde.

Tareas de la práctica

Ejercicio 1. (2.5 puntos) Diseña el algoritmo para calcular el mínimo y el máximo de una lista de tamaño n usando como mucho $3\lfloor n/2 \rfloor$ comparaciones. Justifica por qué este límite en el número de comparaciones. [Pista: analiza por separado qué pasa cuando n

es par o impar.] Muestra el resultado de calcular el mínimo y el máximo del vector $v = (5, 19, 30, 904, 13, 54, 3)$.

Ejercicio 2. (1 punto) Diseña un algoritmo divide y vencerás para calcular el mínimo de una lista de tamaño n . Este algoritmo nos permitiría ejecutar las diferentes comparaciones de forma paralela. Muestra el resultado de calcular el mínimo y el máximo del vector $v = (5, 19, 30, 904, 13, 54, 3)$.

Ejercicio 3. (3.5 puntos) Investiga sobre el problema de la subsecuencia común más larga y proponed un algoritmo de programación dinámica para resolverlo. Demostrad su funcionamiento mediante las secuencias de ejemplo anteriores. Añadid las referencias que habéis usado.

Ejercicio 4. (3 puntos) Implementa un algoritmo voraz para gestionar la memoria caché de forma diferida mediante la estrategia mencionada anteriormente. Demuestra que esta estrategia da un resultado óptimo.

Todos los resultados se presentarán en una memoria sencilla pero bien escrita que se deberá entregar en formato PDF y debidamente formateada. Además, el profesor comunicará con suficiente antelación un día y hora de clase donde se hará una defensa de la entrega. La calificación será APTO o NO APTO. En caso de resultar NO APTO, la calificación de la entrega será 0.